LINE Messaging API

註冊申請教學文件

專業排版與設計期中考

班級： 資管三 B

學號： 413630715

姓名： 王品翔

November 6, 2026

摘要

本文件以繁體中文介紹 LINE Messaging API 的完整申請與設定流程，內容涵蓋 LINE Developers 帳號註冊、Provider 與 Messaging API Channel 的建立、Channel Secret 與 Channel Access Token 的取得與保管、Webhook 的設定與以 ngrok 進行本機測試，最後以 Python 與 Flask 實作一個回聲機器人（Echo Bot）並逐行說明程式碼。讀者依照本文件步驟操作，即可在本機完成第一個可接收並回覆訊息的 LINE Bot。

目錄

1 LINE Messaging API 簡介與應用場景 5

1.1 什麼是 LINE Messaging API 5

1.2 常見應用場景 5

1.3 主要訊息類型 5

2 LINE Developers 帳號註冊與 Channel 建立 7

2.1 登入 LINE Developers Console 7

2.2 建立 Provider 7

2.3 建立 Messaging API Channel 7

2.4 建議的回應設定 8

3 Channel Secret 及 Access Token 取得與設定 9

3.1 Channel Secret 9

3.2 Channel Access Token 9

3.3 以環境變數保管憑證 9

4 Webhook 的設定與測試方式 10

4.1 Webhook 的概念 10

4.2 安裝 Python 套件 10

4.3 使用 ngrok 建立公開網址 10

4.4 在 Console 設定 Webhook URL 11

4.5 常見錯誤排除 11

5 範例程式說明：接收及回覆 LINE 訊息 12

5.1 LINE Bot 範例程式碼 12

5.2 程式碼逐段說明 13

5.3 測試結果 14

5.4 延伸應用 14

6 結論 15

7 本文件 LaTeX 原始碼 16

圖目錄

1.1 LINE Messaging API 運作架構 5

2.1 Messaging API Channel 建立流程 8

表目錄

1.1 LINE Bot 常見應用場景 5

2.1 建立官方帳號時的必填欄位 7

3.1 兩種憑證的比較 9

4.1 Webhook 常見錯誤與解決方式 11

程式目錄

3.1 .env 環境變數設定範例 9
4.1 安裝所需 Python 套件 10
4.2 ngrok 啟動指令 10
5.1 Python app.py 範例 12
7.1 本文件 LaTeX 原始碼 main.tex 16

第 1 章

LINE Messaging API 簡介與應用場景

1.1 什麼是 LINE Messaging API

LINE Messaging API 是 LINE 官方提供給開發者的介面，讓開發者能透過「LINE 官方帳號」與使用者進行雙向溝通。當使用者傳送訊息給官方帳號時，LINE 平台會將事件以 HTTP POST 的方式送到開發者指定的 Webhook 網址；開發者的伺服器處理完後，再呼叫 Messaging API 將回覆訊息送回給使用者。整體運作架構如圖 1.1 所示。

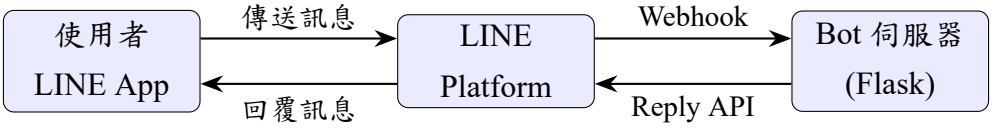


圖 1.1: LINE Messaging API 運作架構

1.2 常見應用場景

Messaging API 能應用在許多實際情境中，表 1.1 整理了常見的應用類型。

表 1.1: LINE Bot 常見應用場景

應用類型	說明
客服機器人	自動回答常見問題，降低人工客服負擔
訂單與預約通知	以推播訊息通知出貨、預約時間或繳費提醒
會員服務	結合 LIFF 與 Rich Menu 提供點數查詢、會員卡
資訊查詢	查詢天氣、課表、公車到站時間等即時資訊
行銷推廣	透過 Flex Message 推送圖文並茂的優惠活動

1.3 主要訊息類型

Messaging API 提供多種回覆方式，最常用的是 `Reply Message`（以 reply token 回覆使用者剛傳來的訊息，免費且不計入額度）與 `Push Message`（主動推播給使用者，會

計入每月訊息額度）。此外還有 Multicast、Broadcast 等大量發送方式，以及 Flex Message、Rich Menu 等豐富的版面元件。

第 2 章

LINE Developers 帳號註冊與 Channel 建立

2.1 登入 LINE Developers Console

步驟 1： 開啟瀏覽器前往 `https://developers.line.biz/`，點選右上角「Log in to Console」。

步驟 2： 使用個人 LINE 帳號登入（可用 QR Code 或電子郵件登入）。

步驟 3： 第一次登入時需建立開發者帳號，輸入名稱與電子郵件並同意開發者條款。

2.2 建立 Provider

Provider 代表提供服務的個人或組織，一個 Provider 底下可以有多個 Channel。在 Console 首頁點選「Create a new provider」，輸入 Provider 名稱（例如自己的名字或專案名稱）後按「Create」即可。

2.3 建立 Messaging API Channel

目前 LINE 的規定是先建立「LINE 官方帳號」，再於官方帳號中啟用 Messaging API。完整流程如圖 2.1 所示。

建立官方帳號時需填寫的欄位整理如表 2.1。

表 2.1: 建立官方帳號時的必填欄位

欄位	說明
帳號名稱	顯示給使用者看的 Bot 名稱，之後可修改
電子郵件	用於接收 LINE 官方通知
公司所在國家或地區	選擇「台灣」
業種（大／小分類）	依用途選擇，例如「個人」或「教育」

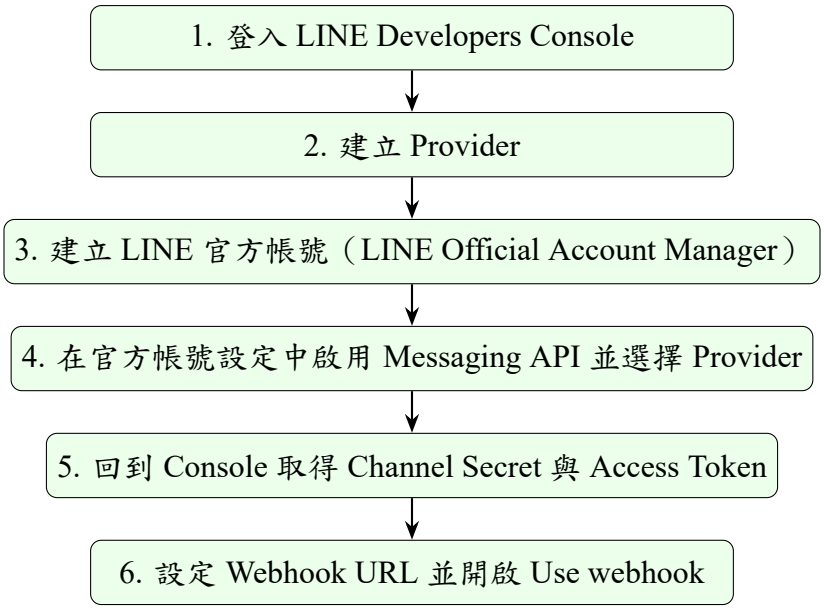


圖 2.1: Messaging API Channel 建立流程

2.4 建議的回應設定

在 LINE Official Account Manager 的「回應設定」中，建議將「聊天」關閉、「Webhook」開啟，並關閉「自動回應訊息」與「加入好友的歡迎訊息」，避免官方預設訊息和自己的程式回覆重複。

第 3 章

Channel Secret 及 Access Token 取得與設定

3.1 Channel Secret

Channel Secret 位於 Channel 頁面的「Basic settings」分頁。它的用途是驗證 Webhook 請求確實來自 LINE 平台：LINE 會用 Channel Secret 對請求內容做 HMAC-SHA256 運算，並放在 `X-Line-Signature` 標頭中，伺服器端再以同樣方式比對。

3.2 Channel Access Token

切換到「Messaging API」分頁，捲動到最下方的「Channel access token (long-lived)」，按下「Issue」即可產生長期有效的 Access Token。呼叫 Reply、Push 等 API 時，都必須在 HTTP 標頭中帶上此 Token。

表 3.1: 兩種憑證的比較

	Channel Secret	**Channel Access Token**
位置	Basic settings 分頁	Messaging API 分頁
用途	驗證 Webhook 簽章	呼叫 Messaging API 的授權
使用端	接收訊息時	送出訊息時
外洩風險	他人可偽造請求	他人可冒用 Bot 發訊息

3.3 以環境變數保管憑證

憑證絕對不能直接寫在程式碼裡或上傳到 GitHub。建議在專案目錄建立 `.env` 檔，並將其加入 `.gitignore`，內容格式如程式碼 3.1。

程式碼 3.1: .env 環境變數設定範例

```
LINE_CHANNEL_SECRET=請填入你的ChannelSecret
LINE_CHANNEL_ACCESS_TOKEN=請填入你的ChannelAccessToken
```

第 4 章

Webhook 的設定與測試方式

4.1 Webhook 的概念

Webhook 是 LINE 平台主動「回呼」開發者伺服器的機制。由於 LINE 平台必須能從網際網路連到你的伺服器，且網址必須是 HTTPS，因此在本機開發時，需要透過 ngrok 將本機埠號轉成公開的 HTTPS 網址。

4.2 安裝 Python 套件

首先安裝範例程式需要的套件，指令如程式碼 4.1。

程式碼 4.1: 安裝所需 Python 套件

```
pip install flask line-bot-sdk python-dotenv
```

4.3 使用 ngrok 建立公開網址

步驟 1： 至 ngrok 官網註冊帳號並下載 ngrok。

步驟 2： 依官網指示設定 authtoken。

步驟 3： 先啟動 Flask 程式（監聽 8000 埠），再另開一個終端機執行程式碼 4.2 的指令。

步驟 4： 複製畫面上 Forwarding 欄位的 `https://xxxx.ngrok-free.app` 網址。

程式碼 4.2: ngrok 啟動指令

```
ngrok config add-authtoken <你的authtoken>
python app.py
ngrok http 8000
```

4.4 在 Console 設定 Webhook URL

回到 Channel 的「Messaging API」分頁，在「Webhook URL」欄位填入 ngrok 網址並加上路徑 `/callback`，例如 `https://xxxx.ngrok-free.app/callback`。接著按「Verify」，若顯示 Success 代表連線成功，最後記得開啟「Use webhook」開關。

4.5 常見錯誤排除

表 4.1: Webhook 常見錯誤與解決方式

錯誤現象	可能原因與解決方式
Verify 回傳 404	網址少了 `/callback` 路徑
Verify 回傳 400	Channel Secret 錯誤導致簽章驗證失敗
Verify 逾時	Flask 未啟動，或 ngrok 埠號與程式不一致
Bot 沒有回覆	未開啟 Use webhook，或 Access Token 錯誤
重開 ngrok 後失效	免費版網址會變動，需重新填寫 Webhook URL

第 5 章

範例程式說明：接收及回覆 LINE 訊息

5.1 LINE Bot 範例程式碼

以下範例使用 Python 與 Flask 框架實作一個簡單的回聲機器人（Echo Bot），會把使用者傳來的文字訊息原封不動地回覆，完整程式碼如程式碼 5.1。

程式碼 5.1: Python app.py 範例

```python
from flask import Flask, request, abort
import os
from linebot.v3 import WebhookHandler
from linebot.v3.messaging import (Configuration, ApiClient, MessagingApi,
                                  ReplyMessageRequest, TextMessage)
from linebot.v3.exceptions import InvalidSignatureError
from linebot.v3.webhooks import MessageEvent, TextMessageContent
from dotenv import load_dotenv

# 載入 .env 中的環境變數
load_dotenv()

app = Flask(__name__)

# 從環境變數讀取 LINE 憑證，避免把金鑰寫死在程式碼中
CHANNEL_SECRET = os.getenv("LINE_CHANNEL_SECRET")
CHANNEL_ACCESS_TOKEN = os.getenv("LINE_CHANNEL_ACCESS_TOKEN")

configuration = Configuration(access_token=CHANNEL_ACCESS_TOKEN)
handler = WebhookHandler(CHANNEL_SECRET)


@app.route("/callback", methods=['POST'])
def callback():
    # 取得 LINE 平台附上的簽章，用來驗證請求來源
    signature = request.headers.get('X-Line-Signature', '')
```

```
27      body = request.get_data(as_text=True)
28      app.logger.info(f"Request body: {body}")
29
30      try:
31          handler.handle(body, signature)
32      except InvalidSignatureError:
33          abort(400)  # 簽章錯誤，代表不是 LINE 送來的請求
34
35      return 'OK'
36
37
38  @handler.add(MessageEvent, message=TextMessageContent)
39  def handle_message(event):
40      # 收到文字訊息時，原封不動回覆給使用者
41      with ApiClient(configuration) as api_client:
42          messaging_api = MessagingApi(api_client)
43          reply = ReplyMessageRequest(
44              reply_token=event.reply_token,
45              messages=[TextMessage(text=event.message.text)]
46          )
47          messaging_api.reply_message_with_http_info(reply)
48
49
50  if __name__ == "__main__":
51      app.run(host='0.0.0.0', port=8000, debug=True)
```

5.2 程式碼逐段說明

- **匯入套件（第 1–8 行）：**匯入 Flask 網頁框架與 LINE Bot SDK v3 的相關類別，並匯入 load_dotenv 讀取環境變數。
- **讀取憑證（第 10–20 行）：**從 .env 取得 Channel Secret 與 Access Token，分別建立 WebhookHandler（負責驗證與分派事件）及 Configuration（負責呼叫 API）。
- **callback 函式（第 23–35 行）：**定義 /callback 路由接收 LINE 的 POST 請求，取出 X-Line-Signature 與請求內容交給 handler 驗證；簽章不符時回傳 HTTP 400，成功則回傳 OK。
- **handle_message 函式（第 38–47 行）：**以裝飾器註冊「文字訊息事件」的處理函式，利用事件中的 reply_token 建立 ReplyMessageRequest，將使用者的文字原樣回覆。

- **啟動伺服器（第 50–51 行）：**在 0.0.0.0 的 8000 埠啟動 Flask，搭配 ngrok 即可對外提供服務。

5.3 測試結果

在 Console 的「Messaging API」分頁掃描 QR Code 將 Bot 加為好友，傳送任何文字，例如「你好」，Bot 就會立刻回覆「你好」，代表整個流程已成功運作。

5.4 延伸應用

完成 Echo Bot 後，可以進一步嘗試以下功能：使用 `push_message` 主動推播通知；建立 Rich Menu 提供常用功能按鈕；或以 Flex Message 設計卡片式的圖文訊息，讓 Bot 的介面更豐富。

第 6 章

結論

本文件從 LINE Messaging API 的基本概念出發，依序說明開發者帳號註冊、Channel 建立、憑證取得與保管、Webhook 設定與 ngrok 測試，最後以 Python 實作並解析一個完整可運作的 Echo Bot。掌握這些步驟後，開發者即可在此基礎上延伸出客服、通知、查詢等各式實用的 LINE 應用。

第 7 章

本文件 LaTeX 原始碼

本章列出產生本文件的完整 LaTeX 原始碼。

程式碼 7.1: 本文件 LaTeX 原始碼 main.tex

```
% !TEX program = xelatex
\documentclass[12pt,a4paper]{report}

% ===== 中文與字型 =====
\usepackage{fontspec}
\usepackage{xeCJK}
% 依照課堂範例 week3-1002 的字型設定：標楷體 + SimSun + Times New Roman
% 標楷體沒有粗體，所以用 AutoFakeBold 產生粗體效果
\IfFontExistsTF{標楷體}{%
  \setCJKmainfont[AutoFakeBold=2, AutoFakeSlant=0.2]{標楷體}
  \setCJKsansfont[AutoFakeBold=2, AutoFakeSlant=0.2]{標楷體}
  \setCJKmonofont[AutoFakeBold=2, AutoFakeSlant=0.2]{SimSun}
  \setmainfont{Times New Roman}
}{%
  % 非 Windows 電腦的備用字型
  \setCJKmainfont[AutoFakeBold=2, AutoFakeSlant=0.2]{Noto Serif CJK TC}
  \setCJKsansfont[AutoFakeBold=2, AutoFakeSlant=0.2]{Noto Sans CJK TC}
  \setCJKmonofont[AutoFakeBold=2, AutoFakeSlant=0.2]{Noto Sans CJK TC}
}
% 中文自動斷行（課堂範例的必要設定）
\XeTeXlinebreaklocale "zh"
\XeTeXlinebreakskip = 0pt plus 1pt
\linespread{1.3}

% ===== 版面與套件 =====
\usepackage[margin=2.5cm]{geometry}
\usepackage{xcolor}
\usepackage{graphicx}
\usepackage{booktabs}
\usepackage{tabularx}
```

```
\usepackage{enumitem}
\usepackage{tikz}
\usetikzlibrary{arrows.meta,positioning,shapes}
\usepackage{listings}
\usepackage{caption}
\usepackage{titlesec}
% 章標題改為「第 X 章」
\titleformat{\chapter}[display]{\bfseries\huge}{第\,\thechapter\,章}{0.6em}{\
    Huge}
\titlespacing*{\chapter}{0pt}{0pt}{1.5em}
\usepackage[hidelinks]{hyperref}

% ===== 中文名稱 =====
\renewcommand{\contentsname}{目錄}
\renewcommand{\listfigurename}{圖目錄}
\renewcommand{\listtablename}{表目錄}
\renewcommand{\figurename}{圖}
\renewcommand{\tablename}{表}
\renewcommand{\chaptername}{第}
\renewcommand{\lstlistingname}{程式碼}
\renewcommand{\lstlistlistingname}{程式目錄}
\renewcommand{\abstractname}{摘要}

% ===== listings 設定：左側自動行號 =====
\definecolor{codebg}{RGB}{247,247,247}
\definecolor{kw}{RGB}{0,0,180}
\definecolor{cm}{RGB}{0,120,0}
\definecolor{str}{RGB}{160,30,30}
\lstset{
  basicstyle=\ttfamily\small,
  numbers=left,
  numberstyle=\tiny\color{gray},
  stepnumber=1,
  numbersep=8pt,
  backgroundcolor=\color{codebg},
  frame=single,
  rulecolor=\color{gray},
  breaklines=true,
  breakatwhitespace=false,
  showstringspaces=false,
```

```
  tabsize=4,
  captionpos=t,
  keywordstyle=\color{kw}\bfseries,
  commentstyle=\color{cm},
  stringstyle=\color{str},
  extendedchars=false,
  columns=fullflexible,
  keepspaces=true
}

\title{\textbf{LINE Messaging API 註冊申請教學文件}\\[1em]\large 專業排版與設計期中考}
\author{資管三B \quad 王品翔 \quad 413630715}
\date{\today}

\begin{document}

% ===== 封面 =====
\begin{titlepage}
  \centering
  \vspace*{3cm}
  {\Huge\bfseries LINE Messaging API\\[0.5em] 註冊申請教學文件\par}
  \vspace{1.5cm}
  {\Large 專業排版與設計 期中考\par}
  \vspace{3cm}
  \begin{tabular}{rl}
    班級： & 資管三B \\[0.5em]
    學號： & 413630715 \\[0.5em]
    姓名： & 王品翔 \\
  \end{tabular}
  \vfill
  {\large \today\par}
\end{titlepage}

% ===== 摘要 =====
\begin{abstract}
本文件以繁體中文介紹 LINE Messaging API 的完整申請與設定流程，內容涵蓋 LINE Developers 帳號註冊、Provider 與 Messaging API Channel 的建立、Channel Secret 與 Channel Access Token 的取得與保管、Webhook 的設定與以 ngrok 進行本機測試，最後以 Python 與 Flask 實作一個回聲機器人（Echo Bot）並逐行說明程式
```

```
    碼。讀者依照本文件步驟操作，即可在本機完成第一個可接收並回覆訊息的 LINE Bot。
\end{abstract}

% ===== 四種目錄 =====
\tableofcontents
\listoffigures
\listoftables
\lstlistoflistings

% ============================================================
\chapter{LINE Messaging API 簡介與應用場景}

\section{什麼是 LINE Messaging API}
LINE Messaging API 是 LINE 官方提供給開發者的介面，讓開發者能透過「LINE 官方帳
    號」與使用者進行雙向溝通。當使用者傳送訊息給官方帳號時，LINE 平台會將事件以
    HTTP POST 的方式送到開發者指定的 Webhook 網址；開發者的伺服器處理完後，再呼叫
    Messaging API 將回覆訊息送回給使用者。整體運作架構如圖~\ref{fig:arch} 所示。

\begin{figure}[htbp]
  \centering
  \begin{tikzpicture}[
      node distance=2.6cm,
      box/.style={draw, rounded corners, minimum width=2.6cm, minimum height
    =1.1cm, align=center, fill=blue!8},
      arr/.style={-{Stealth[length=3mm]}, thick}]
    \node[box] (user) {使用者\\LINE App};
    \node[box, right=of user] (line) {LINE\\Platform};
    \node[box, right=of line] (bot) {Bot 伺服器\\(Flask)};
    \draw[arr] ([yshift=3mm]user.east) -- node[above]{\small 傳送訊息} ([yshift
    =3mm]line.west);
    \draw[arr] ([yshift=3mm]line.east) -- node[above]{\small Webhook} ([yshift
    =3mm]bot.west);
    \draw[arr] ([yshift=-3mm]bot.west) -- node[below]{\small Reply API} ([
    yshift=-3mm]line.east);
    \draw[arr] ([yshift=-3mm]line.west) -- node[below]{\small 回覆訊息} ([
    yshift=-3mm]user.east);
  \end{tikzpicture}
  \caption{LINE Messaging API 運作架構}
  \label{fig:arch}
\end{figure}
```

```latex

\section{常見應用場景}
Messaging API 能應用在許多實際情境中，表~\ref{tab:usecase} 整理了常見的應用類型。

\begin{table}[htbp]
  \centering
  \caption{LINE Bot 常見應用場景}
  \label{tab:usecase}
  \begin{tabularx}{0.9\textwidth}{lX}
    \toprule
    \textbf{應用類型} & \textbf{說明} \\
    \midrule
    客服機器人 & 自動回答常見問題，降低人工客服負擔 \\
    訂單與預約通知 & 以推播訊息通知出貨、預約時間或繳費提醒 \\
    會員服務 & 結合 LIFF 與 Rich Menu 提供點數查詢、會員卡 \\
    資訊查詢 & 查詢天氣、課表、公車到站時間等即時資訊 \\
    行銷推廣 & 透過 Flex Message 推送圖文並茂的優惠活動 \\
    \bottomrule
  \end{tabularx}
\end{table}

\section{主要訊息類型}
Messaging API 提供多種回覆方式，最常用的是 \texttt{Reply Message}（以 reply token 回覆使用者剛傳來的訊息，免費且不計入額度）與 \texttt{Push Message}（主動推播給使用者，會計入每月訊息額度）。此外還有 \texttt{Multicast}、\texttt{Broadcast} 等大量發送方式，以及 Flex Message、Rich Menu 等豐富的版面元件。

% ============================================================
\chapter{LINE Developers 帳號註冊與 Channel 建立}

\section{登入 LINE Developers Console}
\begin{enumerate}[label=步驟 \arabic*：, leftmargin=*]
  \item 開啟瀏覽器前往 \url{https://developers.line.biz/}，點選右上角「Log in to Console」。
  \item 使用個人 LINE 帳號登入（可用 QR Code 或電子郵件登入）。
  \item 第一次登入時需建立開發者帳號，輸入名稱與電子郵件並同意開發者條款。
\end{enumerate}

\section{建立 Provider}
Provider 代表提供服務的個人或組織，一個 Provider 底下可以有多個 Channel。在
```

```
    Console 首頁點選「Create a new provider」，輸入 Provider 名稱（例如自己的名字或專案名稱）後按「Create」即可。

\section{建立 Messaging API Channel}
目前 LINE 的規定是先建立「LINE 官方帳號」，再於官方帳號中啟用 Messaging API。完整流程如圖~\ref{fig:flow} 所示。

\begin{figure}[htbp]
  \centering
  \begin{tikzpicture}[
      node distance=0.55cm,
      stp/.style={draw, rounded corners, fill=green!8, minimum width=8.5cm, minimum height=0.85cm, align=center},
      arr/.style={-{Stealth[length=2.5mm]}, thick}]
    \node[stp] (a) {1. 登入 LINE Developers Console};
    \node[stp, below=of a] (b) {2. 建立 Provider};
    \node[stp, below=of b] (c) {3. 建立 LINE 官方帳號（LINE Official Account Manager）};
    \node[stp, below=of c] (d) {4. 在官方帳號設定中啟用 Messaging API 並選擇 Provider};
    \node[stp, below=of d] (e) {5. 回到 Console 取得 Channel Secret 與 Access Token};
    \node[stp, below=of e] (f) {6. 設定 Webhook URL 並開啟 Use webhook};
    \foreach \x/\y in {a/b,b/c,c/d,d/e,e/f} \draw[arr] (\x) -- (\y);
  \end{tikzpicture}
  \caption{Messaging API Channel 建立流程}
  \label{fig:flow}
\end{figure}

建立官方帳號時需填寫的欄位整理如表~\ref{tab:fields}。

\begin{table}[htbp]
  \centering
  \caption{建立官方帳號時的必填欄位}
  \label{tab:fields}
  \begin{tabularx}{0.9\textwidth}{lX}
    \toprule
    \textbf{欄位} & \textbf{說明} \\
    \midrule
    帳號名稱 & 顯示給使用者看的 Bot 名稱，之後可修改 \\
```

```
    電子郵件 & 用於接收 LINE 官方通知 \\
    公司所在國家或地區 & 選擇「台灣」 \\
    業種（大 / 小分類） & 依用途選擇，例如「個人」或「教育」 \\
    \bottomrule
  \end{tabularx}
\end{table}

\section{建議的回應設定}
在 LINE Official Account Manager 的「回應設定」中，建議將「聊天」關閉、「Webhook」開啟，並關閉「自動回應訊息」與「加入好友的歡迎訊息」，避免官方預設訊息和自己的程式回覆重複。

% ============================================================
\chapter{Channel Secret 及 Access Token 取得與設定}

\section{Channel Secret}
Channel Secret 位於 Channel 頁面的「Basic settings」分頁。它的用途是驗證 Webhook 請求確實來自 LINE 平台：LINE 會用 Channel Secret 對請求內容做 HMAC-SHA256 運算，並放在 \texttt{X-Line-Signature} 標頭中，伺服器端再以同樣方式比對。

\section{Channel Access Token}
切換到「Messaging API」分頁，捲動到最下方的「Channel access token (long-lived)」，按下「Issue」即可產生長期有效的 Access Token。呼叫 Reply、Push 等 API 時，都必須在 HTTP 標頭中帶上此 Token。

\begin{table}[htbp]
  \centering
  \caption{兩種憑證的比較}
  \label{tab:cred}
  \begin{tabularx}{0.95\textwidth}{lXX}
    \toprule
     & \textbf{Channel Secret} & \textbf{Channel Access Token} \\
    \midrule
    位置 & Basic settings 分頁 & Messaging API 分頁 \\
    用途 & 驗證 Webhook 簽章 & 呼叫 Messaging API 的授權 \\
    使用端 & 接收訊息時 & 送出訊息時 \\
    外洩風險 & 他人可偽造請求 & 他人可冒用 Bot 發訊息 \\
    \bottomrule
  \end{tabularx}
\end{table}
```

```
240
241 \section{以環境變數保管憑證}
242 憑證絕對不能直接寫在程式碼裡或上傳到 GitHub。建議在專案目錄建立 \texttt{.env} 檔，
    並將其加入 \texttt{.gitignore}，內容格式如程式碼~\ref{lst:env}。
243
244 \begin{lstlisting}[caption={.env 環境變數設定範例}, label={lst:env}]
245 LINE_CHANNEL_SECRET=請填入你的ChannelSecret
246 LINE_CHANNEL_ACCESS_TOKEN=請填入你的ChannelAccessToken
247 \end{lstlisting}
248
249 % ============================================================
250 \chapter{Webhook 的設定與測試方式}
251
252 \section{Webhook 的概念}
253 Webhook 是 LINE 平台主動「回呼」開發者伺服器的機制。由於 LINE 平台必須能從網際網路
    連到你的伺服器，且網址必須是 HTTPS，因此在本機開發時，需要透過 ngrok 將本機埠
    號轉成公開的 HTTPS 網址。
254
255 \section{安裝 Python 套件}
256 首先安裝範例程式需要的套件，指令如程式碼~\ref{lst:pip}。
257
258 \begin{lstlisting}[language=bash, caption={安裝所需 Python 套件}, label={lst:
    pip}]
259 pip install flask line-bot-sdk python-dotenv
260 \end{lstlisting}
261
262 \section{使用 ngrok 建立公開網址}
263 \begin{enumerate}[label=步驟 \arabic*: , leftmargin=*]
264   \item 至 ngrok 官網註冊帳號並下載 ngrok。
265   \item 依官網指示設定 authtoken。
266   \item 先啟動 Flask 程式（監聽 8000 埠），再另開一個終端機執行程式碼~\ref{lst:
      ngrok} 的指令。
267   \item 複製畫面上 \texttt{Forwarding} 欄位的 \texttt{https://xxxx.ngrok-free.
      app} 網址。
268 \end{enumerate}
269
270 \begin{lstlisting}[language=bash, caption={ngrok 啟動指令}, label={lst:ngrok}]
271 ngrok config add-authtoken <你的authtoken>
272 python app.py
273 ngrok http 8000
```

```
\end{lstlisting}

\section{在 Console 設定 Webhook URL}
回到 Channel 的「Messaging API」分頁，在「Webhook URL」欄位填入 ngrok 網址並加上路徑 \texttt{/callback}，例如 \texttt{https://xxxx.ngrok-free.app/callback}。接著按「Verify」，若顯示 Success 代表連線成功，最後記得開啟「Use webhook」開關。

\section{常見錯誤排除}
\begin{table}[htbp]
  \centering
  \caption{Webhook 常見錯誤與解決方式}
  \label{tab:error}
  \begin{tabularx}{0.95\textwidth}{lX}
    \toprule
    \textbf{錯誤現象} & \textbf{可能原因與解決方式} \\
    \midrule
    Verify 回傳 404 & 網址少了 \texttt{/callback} 路徑 \\
    Verify 回傳 400 & Channel Secret 錯誤導致簽章驗證失敗 \\
    Verify 逾時 & Flask 未啟動，或 ngrok 埠號與程式不一致 \\
    Bot 沒有回覆 & 未開啟 Use webhook，或 Access Token 錯誤 \\
    重開 ngrok 後失效 & 免費版網址會變動，需重新填寫 Webhook URL \\
    \bottomrule
  \end{tabularx}
\end{table}

% ============================================================
\chapter{範例程式說明：接收及回覆 LINE 訊息}

\section{LINE Bot 範例程式碼}
以下範例使用 Python 與 Flask 框架實作一個簡單的回聲機器人（Echo Bot），會把使用者傳來的文字訊息原封不動地回覆，完整程式碼如程式碼~\ref{lst:app}。

\lstinputlisting[language=Python, caption={Python app.py 範例}, label={lst:app}]{app.py}

\section{程式碼逐段說明}
\begin{itemize}[leftmargin=*]
  \item \textbf{匯入套件（第 1--8 行）：} 匯入 Flask 網頁框架與 LINE Bot SDK v3 的相關類別，並匯入 \texttt{load\_dotenv} 讀取環境變數。
```

```
  \item \textbf{讀取憑證（第 10--20 行）：} 從 \texttt{.env} 取得 Channel Secret 與 Access Token，分別建立 \texttt{WebhookHandler}（負責驗證與分派事件）及 \texttt{Configuration}（負責呼叫 API）。
  \item \textbf{callback 函式（第 23--35 行）：} 定義 \texttt{/callback} 路由接收 LINE 的 POST 請求，取出 \texttt{X-Line-Signature} 與請求內容交給 handler 驗證；簽章不符時回傳 HTTP 400，成功則回傳 \texttt{OK}。
  \item \textbf{handle\_message 函式（第 38--47 行）：} 以裝飾器註冊「文字訊息事件」的處理函式，利用事件中的 \texttt{reply\_token} 建立 \texttt{ReplyMessageRequest}，將使用者的文字原樣回覆。
  \item \textbf{啟動伺服器（第 50--51 行）：} 在 0.0.0.0 的 8000 埠啟動 Flask，搭配 ngrok 即可對外提供服務。
\end{itemize}

\section{測試結果}
在 Console 的「Messaging API」分頁掃描 QR Code 將 Bot 加為好友，傳送任何文字，例如「你好」，Bot 就會立刻回覆「你好」，代表整個流程已成功運作。

\section{延伸應用}
完成 Echo Bot 後，可以進一步嘗試以下功能：使用 \texttt{push\_message} 主動推播通知；建立 Rich Menu 提供常用功能按鈕；或以 Flex Message 設計卡片式的圖文訊息，讓 Bot 的介面更豐富。

% ============================================================
\chapter{結論}
本文件從 LINE Messaging API 的基本概念出發，依序說明開發者帳號註冊、Channel 建立、憑證取得與保管、Webhook 設定與 ngrok 測試，最後以 Python 實作並解析一個完整可運作的 Echo Bot。掌握這些步驟後，開發者即可在此基礎上延伸出客服、通知、查詢等各式實用的 LINE 應用。

% ============================================================
\chapter{本文件 LaTeX 原始碼}
本章列出產生本文件的完整 \LaTeX{} 原始碼。

\lstinputlisting[language={[LaTeX]TeX}, caption={本文件 LaTeX 原始碼 main.tex}, label={lst:self}]{main.tex}

\end{document}
```